

Memoria Mi primera app Android

Eloy Jiménez Medina



Índice

1.- Parte A: Análisis del desarrollo móvil	3
1.1.- Ventajas e inconvenientes	3
1.2.- Elección tecnológica	4
1.3.- Ciclo de vida de un activity	5
2.- Parte B: Especificación de la app	6
2.1.- Idea, público y objetivos	6
2.2.- Requisitos funcionales	6
2.3.- Requisitos no funcionales	7
2.4.- Casos de uso	7
2.4.1.- Ficha de usuario 1	8
2.4.2.- Ficha de usuario 2	8
3.- Parte C: Entorno de desarrollo	9
3.1.- Configuración del proyecto	9
3.2.- Dispositivo virtual	9
3.3.- Capturas	9
3.4.- Estructura del proyecto	10
4.- Parte D: Recursos de la aplicación	10
4.1.- Justificación de imágenes	10
4.2.- Justificación de paleta de colores	11
5.- Fuentes y uso de IA	12



1.- Parte A: Análisis del desarrollo móvil

1.1.- Ventajas e inconvenientes

Las ventajas del desarrollo móvil son las siguientes:

- **Movilidad:** El usuario lleva la aplicación siempre consigo en el bolsillo, lo que permite el uso en cualquier lugar y momento.
- **Acceso a funciones nativas de hardware:** Integración fluida con sensores del dispositivo como el GPS, cámara, giroscopio, Bluetooth, entre otros.
- **Notificaciones Push:** La posibilidad de incorporar notificaciones push en la aplicación ofrece un canal directo de comunicación y fidelización con el usuario.
- **Mejor experiencia de usuario:** Las aplicaciones móviles ofrecen interacciones táctiles y mayor fluidez al usuario en comparación con una web.

Sin embargo, el desarrollo móvil también trae numerosos inconvenientes que impactan de lleno en el desarrollo de nuestra aplicación móvil:

- **Batería:** El uso constante de funciones nativas de hardware y la pantalla encendida puede drenar rápidamente la batería, por lo que se debe optimizar las peticiones de funciones nativas de hardware y permitir modos de ahorro de energía.
- **CPU y RAM:** Al ser dispositivos mucho más pequeños que un PC, los dispositivos móviles tienen recursos limitados en comparación con ellos, por lo que se deben optimizar al máximo los procesos para evitar que el sistema operativo cierre la aplicación.
- **Almacenamiento:** El almacenamiento de un teléfono móvil suele ser bastante limitado con respecto al de un PC, por lo que debemos tener muy en cuenta el tamaño que puede llegar a ocupar la APK de nuestra aplicación.
- **Pantalla:** Las pantallas en los dispositivos móviles son pequeñas y de tamaño y resolución variados, por lo que el diseño de nuestra aplicación debe ser adaptable y responsivo para que los elementos clave sean visibles y legibles hasta en las pantallas más pequeñas. También debemos asegurarnos de que todas las áreas de pulsación de nuestra aplicación sean de al menos 48 dp o más.

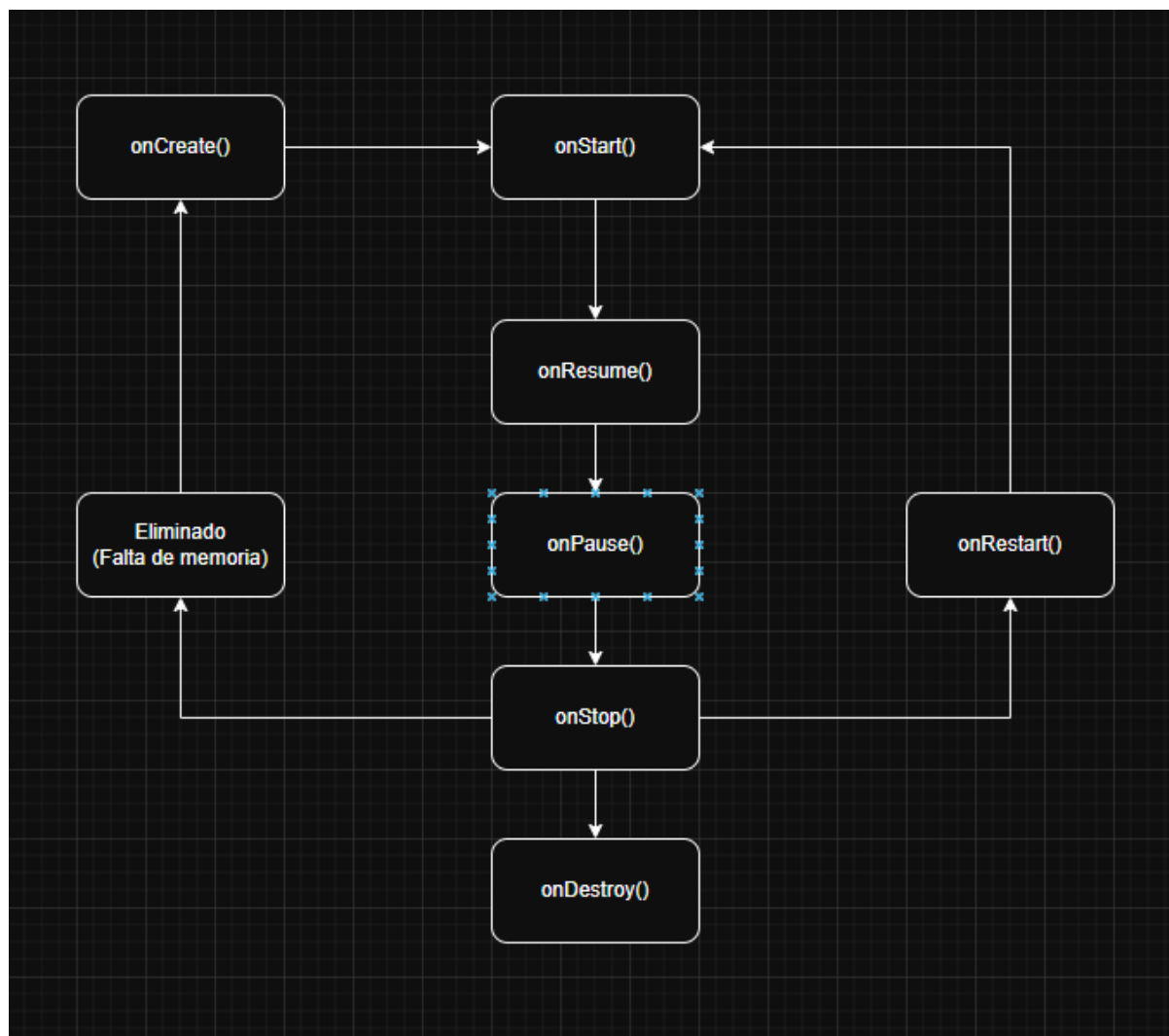


1.2.- Elección tecnológica

	Lenguajes	Rendimiento	Acceso al hardware	Coste	Ejemplos de frameworks
Nativo	Kotlin y Java (Android) Swift y Objective-C (iOS)	Óptimo	Completo	Alto al separar código por plataforma	Android Studio
Híbrido	HTML5, CSS y JavaScript	Moderado	Limitado al requerir plugins de terceros	Bajo al compartirse todo el código	Ionic
Multiplataforma	Dart, JavaScript y TypeScript	Alto	Muy alto debido a la gran variedad de plugins	Medio	Flutter

En este caso, la aplicación se va a desarrollar en Android nativo ya que de esta forma obtendremos un rendimiento superior que con otras opciones ya que la aplicación estará específicamente programada para la plataforma en la que se va a ejecutar. Esto también puede hacer que nuestra aplicación tenga acceso total al hardware sin ningún tipo de limitaciones, al no requerir de plugins extra para acceder a funciones de cámara, GPS, Bluetooth y otros sensores.

1.3.- Ciclo de vida de un activity



- **Si el usuario recibe una llamada:** Se ejecutará `onPause()` (en caso de que la app siga visible parcialmente o en segundo plano) u `onStop()` (en caso de que la llamada ocupe toda la pantalla). Al colgar la llamada, se ejecutarán `onRestart()`, `onStart()` y `onResume()` para que la interfaz vuelva a ser visible.
- **Si el usuario gira el móvil:** Android destruye la activity ejecutando de forma simultánea `onPause()`, `onStop()` y `onDestroy()`. Después de esto, se ejecutarán `onCreate()`, `onStart()` y `onResume()` para crear una nueva instancia adaptada a la orientación de la pantalla.
- **Si el usuario pulsa el botón de volver atrás:** Se ejecutan los métodos de cierre `onPause()`, `onStop()` y `onDestroy()`



2.- Parte B: Especificación de la app

2.1.- Idea, público y objetivos

Mi aplicación es MyTasklist, una aplicación de productividad personal basada en una lista de tareas dirigida principalmente a estudiantes y profesionales que necesitan apuntar rápidamente tareas pendientes del día a día de forma ordenada y sin distracciones. Sus objetivos medibles son:

- Permitir al usuario crear una nueva tarea introduciendo el título y descripción en menos de 10 segundos.
- Visualizar la lista completa de tareas pendientes en la pantalla principal de forma inmediata al abrir la app.
- Asegurar que todos los elementos de la interfaz respondan correctamente a los clics del usuario en cualquier resolución estándar.

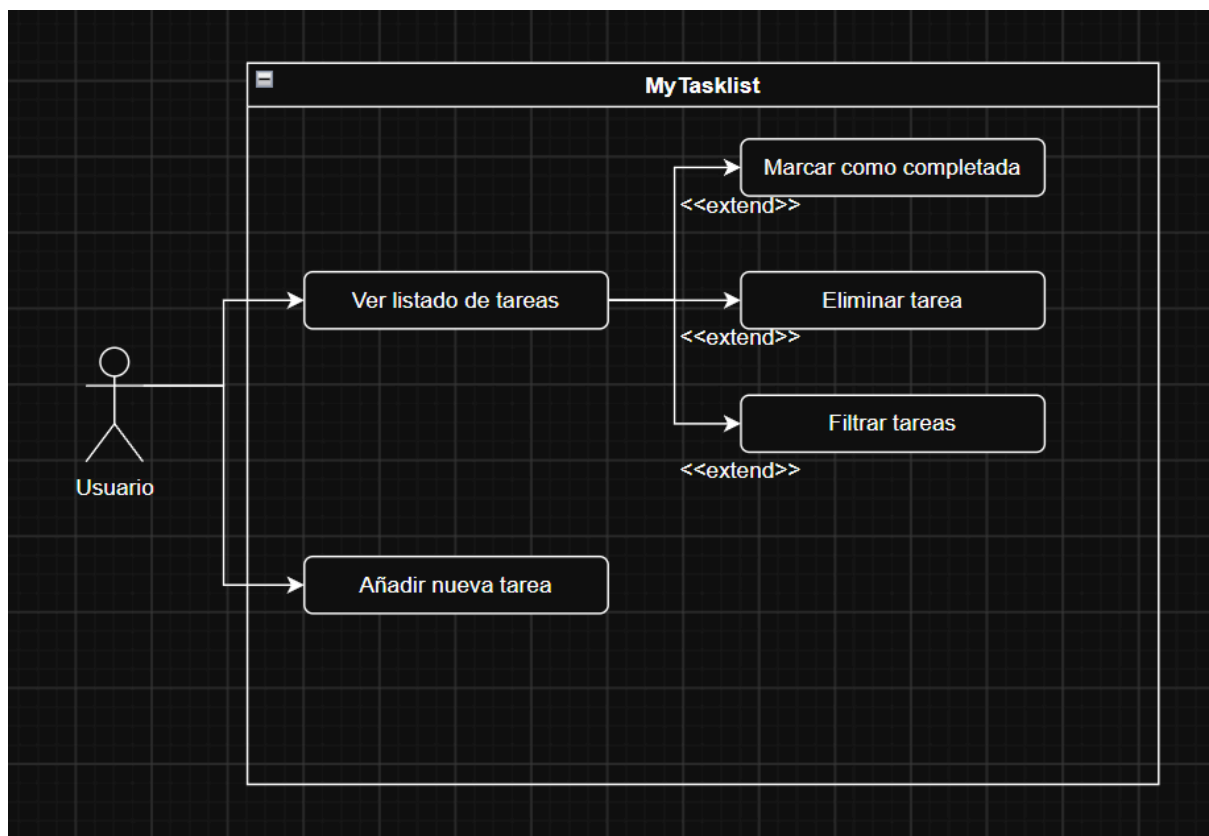
2.2.- Requisitos funcionales

- **RF-01 (Prioridad Alta):** El usuario podrá visualizar un listado con las tareas pendientes en la pantalla principal.
- **RF-02 (Prioridad Alta):** El usuario podrá acceder a un formulario para añadir una nueva tarea.
- **RF-03 (Prioridad Media):** El usuario podrá marcar una tarea como completada.
- **RF-04 (Prioridad Media):** El usuario podrá eliminar una tarea de la lista.
- **RF-05 (Prioridad Baja):** El usuario podrá filtrar las tareas por estado (Pendientes / Completadas).

2.3.- Requisitos no funcionales

- **RNF-01 (Rendimiento):** La pantalla principal cargará el listado de tareas en menos de 3 segundos tras iniciar la aplicación.
- **RNF-02 (Usabilidad):** El tamaño mínimo de los botones táctiles será de 48x48dp.
- **RNF-03 (Compatibilidad):** La aplicación será compatible con dispositivos a partir de Android 7.0 (API 24 - Nougat) para garantizar cobertura al 99,3% de los usuarios.
- **RNF-04 (Fiabilidad):** La app validará los campos obligatorios al crear una tarea para evitar que se guarden registros vacíos.

2.4.- Casos de uso





2.4.1.- Ficha de usuario 1

Precondición: El usuario quiere eliminar una tarea de la lista de tareas.

Flujo principal:

1. El usuario pulsa el botón de Ver listado de tareas.
2. El sistema abre la activity de Ver listado de tareas
3. El usuario presiona el botón de eliminar una tarea
4. El usuario selecciona la tarea que quiere eliminar
5. El sistema elimina la tarea seleccionada

Flujo alternativo: Si la lista de tareas no tiene nada, el sistema mostrará al usuario un mensaje informando que no hay tareas que borrar.

Postcondición: La tarea es eliminada.

2.4.2.- Ficha de usuario 2

Precondición: El usuario quiere crear una tarea nueva.

Flujo principal:

1. El usuario pulsa el botón de nueva tarea.
2. El sistema abre la activity de creación de tareas.
3. El usuario introduce el título, descripción y pulsa el botón de guardar
4. El sistema valida los datos introducidos y los añade a la lista de tareas.
5. Se regresa a la activity anterior.

Flujo alternativo: Si el título está vacío, el sistema mostrará un mensaje de error y no permitirá guardar.

Postcondición: La tarea aparecerá registrada en la lista de tareas.



3.- Parte C: Entorno de desarrollo

3.1.- Configuración del proyecto

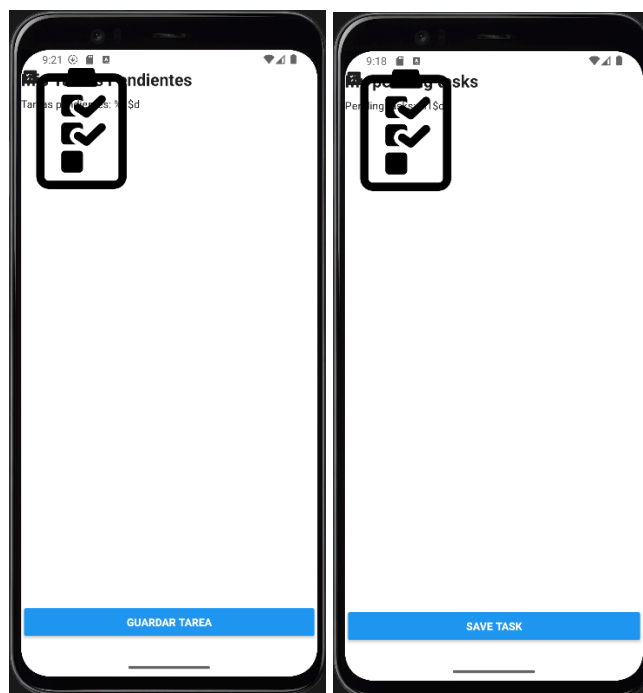
- **Nombre:** PMDM_P1_Eloyjimenez
- **Paquete:** es.medac.eloy.app
- **Plantilla:** Empty Views Activity con API 24 como mínimo

3.2.- Dispositivo virtual

Dispositivo	Resolución	Densidad	API
Pixel 4	1080x2280px	440 dpi	34

3.3.- Capturas

Pixel 4 en español y en inglés





3.4.- Estructura del proyecto

- **AndroidManifest.xml:** Este fichero xml es el que declara las actividades de la app y la configuración general del proyecto. Sin el, no hay aplicación Android que funcione
- **java/:** Contiene el código de Java de cada pantalla.
- **res/:** Contiene los recursos visuales del proyecto, como los strings, paleta de colores o estilos. En él es donde se tienen que definir las variables de diseño para evitar el hardcoding.
- **Ficheros Gradle:** Estos ficheros controlan las dependencias, plugins y configuraciones de compilación de SDK.
- **Relación entre Activity (Java) y su Layout (XML):** La Activity (java) vincula su diseño con setContentView conectando la lógica de Java con los componentes visuales establecidos en el XML.

4.- Parte D: Recursos de la aplicación

4.1.- Justificación de imágenes

Las imágenes vectoriales conviene usarlas para los iconos de la interfaz ya que estas pueden reescalarsen sin distorsión y ocupan el mínimo espacio de almacenamiento al no necesitar versiones de diferentes tamaños.

Conviene usar las imágenes de mapa de bits para imágenes fotográficas y complejas como por ejemplo las fotos de perfil, catálogo de productos o imágenes de fondo ya que en estos casos los vectores no bastan para renderizar la imagen.



4.2.- Justificación de paleta de colores

- **Color primario #2196F3 (Azul):** Utilizado para resaltar los elementos más importantes de la aplicación como los botones de acción. Se ha escogido el azul ya que este color suele estar ligado a la eficiencia, la confianza y la concentración, algo fundamental en una aplicación de tareas.
- **Color secundario #BBDEFB (Azul claro):** Este color se utiliza para los elementos secundarios, bordes y elementos seleccionados. He elegido el azul claro para mantener coherencia visual en la aplicación con respecto al color primario.
- **Color fondo #FFFFFF (Blanco):** Para el fondo he elegido el color blanco para evitar la fatiga visual al usuario y proveer una interfaz limpia y minimalista
- **Color texto #212121 (Gris oscuro):** He decidido aplicar un tono de gris oscuro para el texto para que se diferencie bien de los colores primario, secundario y fondo, haciendo que el texto sea legible. No elegí el color negro para evitar la fatiga visual que puede provocar el contraste entre el negro y blanco puro.
- **Color tarea completada #4CAF50 (Verde):** He elegido este color para las tareas ya terminadas ya que el verde suele estar relacionado con el éxito y la aprobación.



5.- Fuentes y uso de IA

La gran mayoría de la documentación se ha obtenido de los recursos disponibles del aula virtual (como los temarios y cheat sheet) e intuición propia en el caso de las ventajas del desarrollo móvil, los requisitos (tanto funcionales como no funcionales), casos de uso y paleta de colores y estructura de proyecto. El uso de IA se ha limitado principalmente en solucionar errores de compilación en el proyecto como el que describe en uno de los commits de GitHub.

Las principales fuentes externas de información que he utilizado han sido las siguientes:

(S/f). Seidor.com. Recuperado el 8 de octubre de 2026, de

<https://www.seidor.com/es-es/blog/tipologias-desarrollo-movil-nativo-hibrido-cross-platform>

Comparación entre mapa de bits de imagen y vector de imagen. (s/f). Android Developers.

Recuperado el 8 de octubre de 2026, de

<https://developer.android.com/develop/ui/compose/graphics/images/compare?hl=es-419>

Cortázar, R. (2023, julio 14). Mapa de bits y vectores, ¿qué son y cuál elijo? Blog de

Acumbamail. <https://acumbamail.com/blog/mapa-de-bits-y-vectores/>

Imagen vectorial y mapa de bits frente a frente. (2020, julio 20). ionos Digital Guide.

<https://www.ionos.com/es-us/digitalguide/paginas-web/disenio-web/imagen-vectorial-o-mapa-de-bits-pros-y-contras/>

Qualisys. (2024, agosto 27). Desarrollo de aplicaciones móviles: tendencias y herramientas.

Qualisys.

<https://www.qualisys.es/desarrollo-de-aplicaciones-moviles-tendencias-y-herramientas/>